

IntentTester: Intent-Driven Multi-agent Framework for Cross-Library Test Migration

YI GAO, Zhejiang University, China and the Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China

ZIYUAN ZHANG, Zhejiang University, China

XING HU, Zhejiang University, China

XIAOHU YANG, Zhejiang University, China

XIN XIA*, Zhejiang University, China and the Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China

Unit tests capture both functional checks and domain-specific knowledge, but this knowledge remains locked within individual projects and is rarely reused across libraries with overlapping functionality. Existing migration techniques based on structural code mappings (e.g., API signatures) often break down under divergent designs or cross-language settings, resulting in non-executable migrated tests. In this paper, we present INTENTTESTER, a multi-agent framework for intent-driven test reuse. Instead of translating raw code, INTENTTESTER abstracts tests into a language-agnostic Test Description Language (TDL), aligns them with semantically related entities and dependencies in a repository graph, and synthesizes executable tests through LLM-guided reasoning and iterative validation. This design enables cross-library and cross-language migration without manual intervention, producing migrated tests that existing structure-mapping approaches cannot achieve. We evaluate INTENTTESTER on nine open-source projects across three domains (JSON, HTML, and Time) and two languages (Java and Python). INTENTTESTER generates 2,776 syntactically correct tests with 85% correctness; in comparison, the two baselines achieve 51% and 43%. Among them, 2,410 tests executed successfully, yielding a 74% effectiveness rate. Beyond higher success rates, INTENTTESTER also surfaced previously unknown defects—including stack overflows, null dereferences, and parsing inconsistencies, several of which have been acknowledged or patched by maintainers. Our results show that intent-driven migration shifts the focus from code mappings to semantic alignment, allowing practical cross-library and cross-language test reuse while improving test quality and exposing implementation flaws.

CCS Concepts: • **Software and its engineering** → **Software evolution**; **Reusability**.

Additional Key Words and Phrases: Test Reuse, Intent-driven Migration, Repository Graph, Large Language Model

ACM Reference Format:

Yi Gao, Ziyuan Zhang, Xing Hu, Xiaohu Yang, and Xin Xia. 2026. IntentTester: Intent-Driven Multi-agent Framework for Cross-Library Test Migration. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE068 (July 2026), 21 pages. <https://doi.org/10.1145/3797096>

*Corresponding Author

Authors' Contact Information: Yi Gao, Zhejiang University, Hangzhou, China and the Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, China, gaoyi01@zju.edu.cn; Ziyuan Zhang, Zhejiang University, Hangzhou, China, ziyuanzhang@zju.edu.cn; Xing Hu, Zhejiang University, Hangzhou, China, xinghu@zju.edu.cn; Xiaohu Yang, Zhejiang University, Hangzhou, China, yangxh@zju.edu.cn; Xin Xia, Zhejiang University, Hangzhou, China and the Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, China, xin.xia@acm.org.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE068

<https://doi.org/10.1145/3797096>

1 Introduction

Unit testing is the backbone of software reliability, and open-source communities such as GitHub now host millions of high-quality unit tests created by developers to validate a wide range of functionalities [15, 16, 31, 39, 44, 47, 48]. These tests not only verify functional correctness but also embody valuable domain knowledge. However, this rich body of testing knowledge is locked inside individual projects, rarely reused across libraries that offer similar functionality [25, 39, 42, 45]. When multiple libraries implement the same functionality, developers end up writing tests from scratch even though equivalent tests already exist elsewhere. For example, JSON parsing has both *Gson* (Java) and *SimpleJson* (Python), while HTML parsing has *Jsoup* and *JFfiveparse* (both in Java). In practice, these overlapping domains mean developers duplicate effort, slow down the adoption of new libraries, and risk leaving defects undiscovered when test suites are inconsistently reimplemented [21].

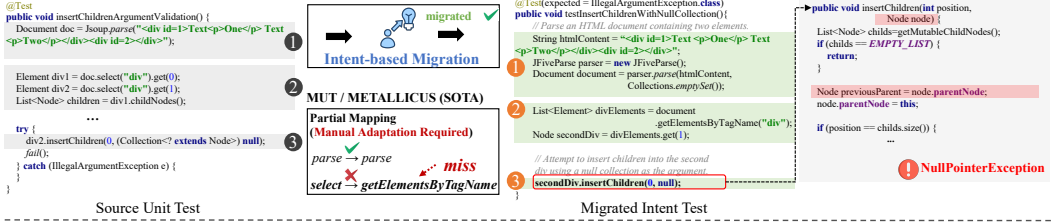
A natural solution is cross-library test migration, which aims to reuse tests across libraries with similar functionality. However, current approaches such as MUT [21] and METALLICUS [28] rely on structural code mappings, aligning API signatures or code patterns between libraries and then translating the test code accordingly. This mapping-first paradigm suffers from two fundamental limitations: First, *structural and linguistic heterogeneity*. Libraries that provide similar features often expose them through divergent API designs, coding styles, or even across different programming languages, making structural mappings sparse or infeasible. Second, *manual adaptation overhead*. Even when partial mappings exist, state-of-the-art tools typically require developers to manually adjust the generated test before it can run, introducing extra costs and limiting automation. As a result, most prior tools struggle to deliver runnable tests in diverse or cross-language settings, limiting their ability to fully leverage the potential of test knowledge reuse. To enable cross-library test reuse and facilitate the sharing of best practices, we identify two core challenges:

Challenge 1: Structural incompatibility. Even when libraries implement the same functionality, their internal code structures and API designs diverge significantly. Direct structural mappings are scarce, and differences in design philosophy, coding styles, or language constructs make API-level alignment unreliable.

Challenge 2: Dependency completeness. Executable tests rarely depend on a single API alone. They require constructing fixtures, initializing parameters, and chaining return values to form valid execution paths. When these transitive dependencies are not captured, migrated tests fail at runtime due to missing initializations or parameter mismatches.

To overcome these limitations, we present INTENTTESTER, a multi-agent framework for intent-driven test migration. The key idea is to abstract tests based on their intent, concentrating on the functionality being validated, rather than on raw code or API structures. INTENTTESTER decomposes test migration into five collaborating agents: ❶ **Intent Abstractor Agent** converts a source test into a language-agnostic Test Description Language (TDL), capturing metadata, inputs, execution steps, and assertions. ❷ **Intent Alignment Agent** maps each step in the TDL onto semantically related entities in the target repository graph, expanding dependencies to form a compact but complete context bundle. ❸ **Planning Agent** evaluates whether the retrieved context is sufficient to complete the test as described in the TDL, rejecting any test that lacks the necessary dependencies. ❹ **Test Migration Agent** constructs tests by synthesizing the TDL intent with the relevant context and reference patterns, ensuring alignment with the target repository's behavior. ❺ **Verification Agent** validates the outputs of the previous agents, checking for consistency and sufficiency, and applies lightweight feedback when errors or omissions are detected. This design shifts the problem from brittle code translation to semantic alignment and reasoning, enabling test intents to be

Java-to-Java Cross-Library Test Migration (Source: Jsoup → Target: JFiveParse)



Python-to-Java Cross-Language Test Migration (Source: SimpleJSON → Target: NanoJSON)

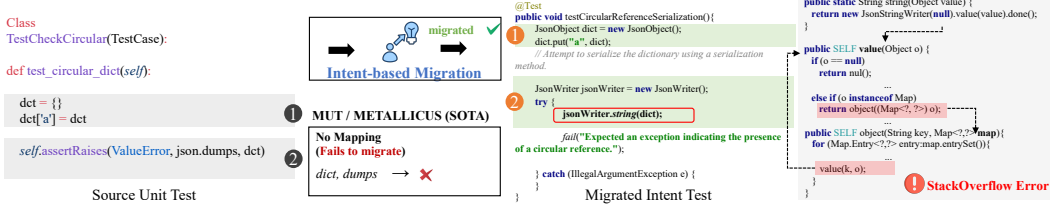


Fig. 1. Examples of cross-library and cross-language test migration. The HTML case (top) contrasts *Jsoup* and *JFiveParse* in handling null insertions, while the JSON case (bottom) shows a circular reference test migrated from *SimpleJSON* to *NanoJSON*. These cases demonstrate that intent-based migration preserves functionality and reveals hidden defects even when explicit structural mappings are incomplete or absent.

migrated across libraries and across programming languages without manual intervention, thus broadening the scope of reusable tests.

We evaluate INTENTTESTER on nine real-world open-source projects across three domains and two programming languages. From 2,058 source tests, we generate 5,536 sub-tests, of which 3,257 remain after filtering. Among these, INTENTTESTER synthesizes 2,776 syntactically correct tests, achieving 85% correctness—well above MUT (51%) and METALLICUS (43%). Out of the correct tests, 2,410 execute successfully in the target repositories, corresponding to a 74% effectiveness rate. Beyond execution success, INTENTTESTER also uncovered 25 real defects across JSON, HTML, and Time libraries, including nested JSON parsing failures, missing HTML validation, and stack overflows in recursive serialization. Several of these issues have already been acknowledged or patched by maintainers. These results confirm that intent-driven migration not only surpasses code-mapping baselines in producing runnable tests but also provides actionable value by strengthening test suites and revealing latent defects in widely used libraries.

The main contributions of this paper are as follows:

- We identify the limitations of code-mapping approaches to test reuse and introduce intent-driven test migration as a new paradigm.
- We present INTENTTESTER, a multi-agent approach that leverages TDL abstraction, repository graph reasoning, and LLM-guided synthesis to enable cross-library test migration, with a replication package available at [5].
- We build a fully automated pipeline supporting both Java and Python libraries, capable of generating runnable tests without manual adaptation. By decoupling migration from explicit mappings, it broadens reuse opportunities previously missed by prior approaches.
- We conduct a large-scale study on nine repositories, showing that INTENTTESTER improves syntactic correctness by 30–40% and execution success by over 20% compared to baselines, while uncovering 25 real defects.

2 Motivation Example

Figure 1 presents two concrete intent-based test migration examples from real-world GitHub projects. *Jsoup* and *JFiveParse* are both Java libraries designed for HTML parsing and manipulation. In *Jsoup*, the test reveals built-in argument validation that throws an `IllegalArgumentException` when attempting to insert null as a child node. This design ensures that client projects using *Jsoup* must handle the exception appropriately, thereby preventing unintended null insertions.

When this test is migrated to *JFiveParse*, the same test intent is expressed through a newly migrated test. However, unlike *Jsoup*, *JFiveParse* only validates the parent node and omits checks on the inserted child nodes. As a result, executing `secondDiv.insertChildren(0, null)` dereferences a missing parent pointer, triggering a `NullPointerException`. If this issue occurs in a client project relying on *JFiveParse*, it could lead to an unexpected program crash.

This example highlights the limitations of state-of-the-art migration approaches such as MUT and METALLICUS, which rely on structural API mapping. While they can align basic calls like `parse` and `insertChildren`, they miss `select`, leading to **incomplete mappings** and requiring manual adaptation to make the migrated test executable. However, by abstracting the source test into a language-agnostic intent, our approach achieves completeness in migration, automatically covering cases where structural mappings are partial or absent. This enables the generation of fully executable tests without manual intervention.

Another example involves JSON serialization libraries across different languages. As shown in the lower-left part of Figure 1, *SimpleJSON* (Python) includes a dedicated test for handling circular references, where a dictionary key points to the dictionary itself. The test intent can be summarized as: detect circular references and raise an exception, ensuring that client projects cannot serialize self-referencing objects without explicit handling. In contrast, *NanoJSON* (Java) offers lightweight JSON serialization. Migrating the test intent to *NanoJSON* produces a corresponding unit test that attempts to validate the same behavior. However, when executing the intent test in *NanoJSON*, the project terminates with a `StackOverflowError`. After further analysis, we identify that *NanoJSON*'s `JsonWriterBase` processes Map objects by iterating over each key-value pair and recursively invoking the `value()` method on the values. In the presence of a self-referencing object (e.g., `dict.put("a", dict)`), the serialization process enters an infinite recursion: `value(dict) → object(map) → value("a", dict) → object(map) → value("a", dict)`, ultimately leading to a runtime crash. This behavior represents a significant risk: if a client project using *NanoJSON* inadvertently introduces a circular reference—or, more critically, if an attacker exploits this defect—the client crashes due to unhandled recursion.

As shown in Figure 1, this case poses a critical challenge: there is **no structural mapping** between the dumps API and Java serialization entry points. Unlike structure-based mapping, our approach aligns tests at the intent level, enabling the reuse of Python test intents in Java libraries even when APIs share no structural commonality, thereby surpassing the structural limitations of prior tools. This design allows us to migrate tests across both libraries and languages, and in practice, it exposes hidden defects such as unhandled recursion in *NanoJSON*.

We report the issues identified through intent-based testing to the respective library maintainers. Both *NanoJSON* and *JFiveParse* authors confirm the validity of our findings. Notably, the maintainers of *JFiveParse* have already addressed the issue by adding null checks to their repository, effectively mitigating the NPE risk.

3 Approach

Our approach consists of a preprocessing step (sec. 3.1) that builds a repository graph for the target repository, followed by a multi-agent framework `INTENTTESTER` for intent-driven test

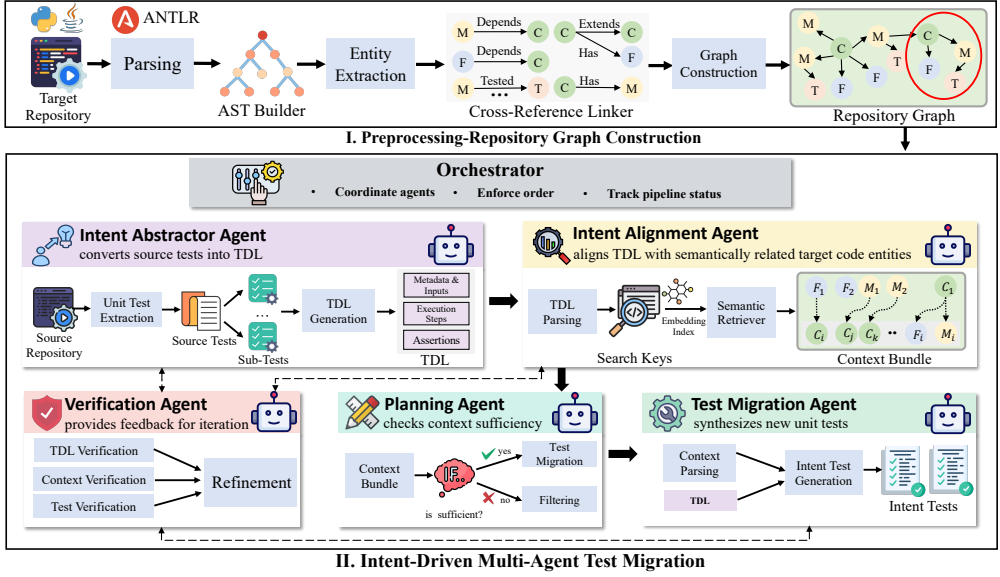


Fig. 2. Overall Pipeline Intent-Driven Test Migration.

Table 1. Edge types in the repository graph, capturing structural and behavioral relations among classes, methods, fields, and tests.

Edge Type	Scope	Description and Example
INHERITS	Class $\leftrightarrow$ Class	Class extends a superclass (e.g., <code>JsonObject</code> $\rightarrow$ <code>JsonElement</code>).
IMPLEMENTS	Class $\leftrightarrow$ Class	Class implements an interface (e.g., <code>Moment</code> $\rightarrow$ <code>UnixTime</code>).
HAS_METHOD	Class $\leftrightarrow$ Method	Method declared in a class (e.g., <code>JsonParser.parseString</code>).
HAS_FIELD	Class $\leftrightarrow$ Field	Field declared in a class (e.g., <code>JsonArray.elements</code>).
CALLS_METHOD	Method $\leftrightarrow$ Method	One method calls another (e.g., <code>parseString</code> $\rightarrow$ <code>readToken</code>).
ACCESSES_FIELD	Method $\leftrightarrow$ Field	Method reads/writes a field (e.g., <code>iterator</code> accesses <code>elements</code>).
DEPENDS_ON_CLASS	Method/Field $\leftrightarrow$ Class	Parameter or field type depends on a class (e.g., <code>parse</code> depends on <code>JsonReader</code>).
RETURNS_CLASS	Method $\leftrightarrow$ Class	Method return type relation (e.g., <code>deepCopy</code> returns <code>JsonElement</code>).
TESTS_METHOD	Test $\leftrightarrow$ Method	Unit test validates a method (e.g., <code>testParse</code> $\rightarrow$ <code>parse</code>).
USES_CLASS	Test $\leftrightarrow$ Class	Test constructs/initializes a class (e.g., <code>new JsonArray()</code>).
ASSERTS_FIELD	Test $\leftrightarrow$ Field	Test checks a field state (e.g., <code>assert size of elements</code>).

migration. The framework involves five agents: the Intent Abstractor (TDL conversion, sec. 3.3), Intent Alignment (semantic retrieval, sec. 3.4), Planning (context sufficiency, sec. 3.5), Test Migration (test synthesis, sec. 3.6), and Verification (validation and feedback, sec. 3.7). An Orchestrator coordinates these agents to ensure a coherent and reliable workflow.

3.1 Preprocessing

In the preprocessing step, we extract code entities from the target repository to construct a corresponding repository graph that supports intent-based alignment. This step is performed once and can be subsequently reused for multiple intent-based test migrations.

3.1.1 Code Entity Extraction. The structure of a software repository and the relationships among its code entities are typically intricate [35]: classes participate in inheritance hierarchies, methods invoke external modules, fields reference user-defined types, and unit tests often interact with multiple interconnected components. Such complexity makes naive parsing insufficient for supporting intent-based test migration, as incomplete or imprecise extraction can lead to missing dependencies and invalid migrated tests.

To address these challenges, we design a structured extraction pipeline that captures fine-grained entities and their structural dependencies. Specifically, we extract four categories of entities—classes, fields, methods, and unit tests—and explicitly link them through inheritance, type references, method invocations, and test-to-method relations. This explicit modeling ensures that dependencies such as parameter types and indirect invocations are preserved, enabling downstream agents to reason over a complete and semantically consistent view of the repository. Technically, we leverage ANTLR [1], an extensible parser framework, to process source code and generate Abstract Syntax Trees (ASTs). We then traverse the ASTs to extract entity definitions and cross-entity references, enriching each entity with metadata such as type signatures, dependencies, and file locations. The pipeline currently supports Java and Python, and can be extended to other ecosystems via ANTLR’s multi-language support.

3.1.2 Graph building and relationship modeling. After extracting code entities, we organize them into a repository graph that encodes their relationships, enabling downstream agents to trace multi-hop dependencies essential for intent-driven test migration.

Algorithm 1: Agent Orchestration for Intent-Driven Test Migration

Input: *sourceTest*: a test from source repo, *targetGraph*: repository graph of target repo
Output: *migratedTest*: executable test or $\emptyset$ if migration fails

```

1 Function CoordinateAgents(sourceTest, targetGraph):
2   tcl  $\leftarrow$  IntentAbstractAgent.extract(sourceTest);
3   context  $\leftarrow$  IntentAlignmentAgent.retrieve(tcl, targetGraph);
4   if Planning.isSufficient(tcl, context) = False then
5     context  $\leftarrow$  IntentAlignment.refine(tcl, targetGraph);
6     if Planning.isSufficient(tcl, context) = False then
7       return  $\emptyset$ 
8     end
9   end
10  candidateTest  $\leftarrow$  TestMigrationAgent.synthesize(tcl, context);
11  result  $\leftarrow$  VerificationAgent.validate(candidateTest);
12  if result = pass then
13    return candidateTest;
14  end
15  else
16    context  $\leftarrow$  VerificationAgent.feedback(tcl, context);
17    return CoordinateAgents(sourceTest, targetGraph);
18  end

```

We define the edge types summarized in Table 1, which capture both structural and behavioral relationships among code entities. These include inheritance hierarchies, method invocations, type dependencies, field accesses, and test-to-method links obtained by statically resolving unit test calls to their target methods. By covering both structural (e.g., *HAS_METHOD*, *DEPENDS_ON_CLASS*)

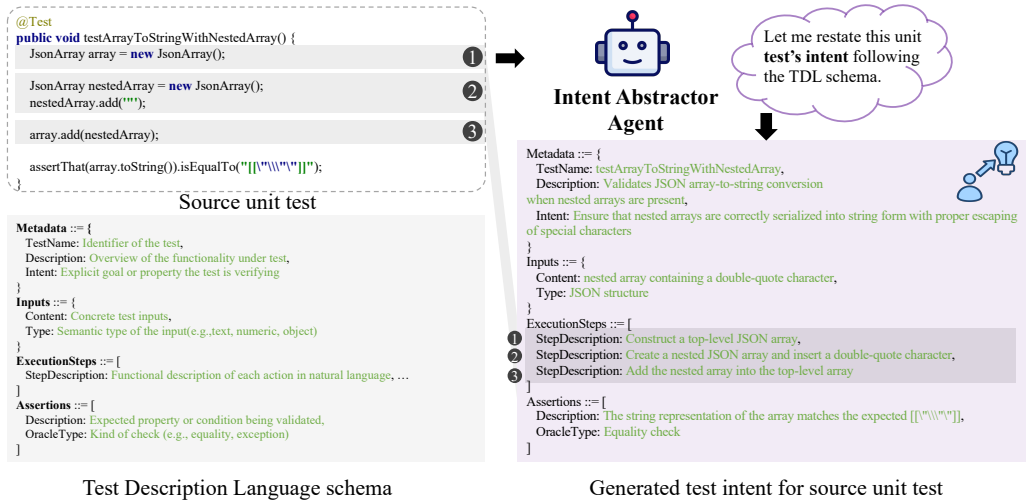


Fig. 3. Example of transforming a source unit test into its TDL representation, capturing metadata, inputs, execution steps, and assertions.

and behavioral (e.g., *CALLS_METHOD*, *TESTS_METHOD*) links, the graph captures repository semantics comprehensively.

Modeling only call graphs or inheritance hierarchies omits important interactions necessary for test reuse, such as indirect dependencies across classes. To address this, we explicitly define a schema that integrates multiple relationship types (structural and behavioral) into a unified representation. This ensures that entities such as tests, methods, and fields are connected through all relevant dependencies, rather than isolated by a single view of the repository. For efficient storage and querying, we use *Neo4j* [11], where entities are represented as labeled nodes and relationships as typed edges, enabling scalable retrieval of subgraphs relevant to a given test intent.

To further support intent-based alignment, each node is enriched with a concise textual description of its functionality. Since comments are often incomplete or outdated, we employ an LLM-based summarization strategy: given each entity's signature and local context (e.g., method body or class definition), the model generates a short natural-language description (e.g., *Creates a deep copy of this element and all its children* for *deepCopy*). These summaries serve as semantic annotations of structural entities and directly support the alignment of test intents with repository nodes (see Sec. 3.4).

3.2 Intent-Driven Multi-Agent Test Migration

3.2.1 Orchestration and Workflow Control. Cross-library test reuse faces challenges from language differences, library designs, and testing logic. To address these, we decompose the task into specialized agents coordinated by an Orchestrator. As shown in Algorithm 1, the Orchestrator first invokes the *Intent Abtractor Agent* to derive the TDL representation, then calls the *Intent Alignment Agent* to retrieve related context, and the *Planning Agent* to assess the sufficiency of the retrieved context. Finally, the *Test Migration Agent* synthesizes executable tests, while the *Verification Agent* validates the result and, if necessary, feeds corrections back into the pipeline.

3.3 Test Intent Abstraction

The *Intent Abtractor Agent* transforms source tests into a unified representation by first decomposing complex tests into smaller units and then converting each unit into a TDL encoding its functional intent.

3.3.1 Test Splitting. Source tests are often multi-purpose, combining transferable logic (e.g., parsing) with library-specific mechanisms (e.g., error handling), which reduces reusability. To enable fine-grained reuse and improve the robustness of later agents, we decompose multi-purpose tests into simpler sub-tests, each aligned with a single functional intent.

Unlike prior work on test decomposition [22], which focused on analyzing test smells, our splitting procedure is redesigned for intent abstraction. First, splitting is tightly integrated with TDL generation: each sub-test is immediately converted into a TDL unit, guaranteeing direct usability for downstream agents. Second, the rules are extended to support both Java and Python tests, enabling cross-language consistency. Third, splitting improves fault isolation: if one sub-test is incorrectly migrated or fails during execution, the error is localized rather than invalidating the entire original test. Moreover, subsequent agents provide additional safeguards—Planning assesses the adequacy of retrieved context and Verification checks semantic correctness—so potential errors introduced during splitting do not propagate unchecked.

3.3.2 TDL Generation. The TDL captures the core functional intent of a test in a structured, language-agnostic format, enabling reuse across repositories that share functionality but differ in implementation details, programming language, or API design. Traditional tools such as MUT rely on structural mappings, which fail when similar behaviors are expressed through divergent code structures (Figure 1). In contrast, TDL abstracts away implementation details and encodes only the essential test intent, allowing migration even in the absence of explicit mappings.

As shown in Figure 3, a TDL consists of four components: test data, test setup, focal method, and assertions. Given a source test and the predefined schema, the *Intent Abtractor Agent* transforms the test into its TDL using a prompt template that guides the model to restate each case according to the schema. In this example, the test that verifies the serialization of nested JSON arrays is represented as: (i) creating nested arrays, (ii) serializing them, and (iii) asserting correctness of the string output. Such representations are both descriptive and reusable, supporting intent-based retrieval from the target repository graph in subsequent steps. By uniformly applying TDL to all source tests, we decouple test intent from repository-specific implementation, thereby enabling effective cross-library and cross-language migration beyond the reach of structure-mapping approaches. Due to space limitations, we omit the full prompt design, but all templates are provided in our replication package for transparency and reproducibility [5].

3.4 Semantic Context Alignment

Once the source test is abstracted into TDL, the challenge lies in identifying the target repository entities that can fulfill the described functionality. A single test intent often spans multiple classes, methods, and fields, and only complete dependency coverage enables executable test synthesis.

We highlight three key challenges: (i) each TDL description may correspond to multiple candidate entities under cross-language or stylistic variations; (ii) capturing only the most similar node is insufficient without its dependencies such as parameters, return values, and invocation chains; and (iii) large amounts of unrelated code reduce generation quality, making it essential to construct a minimal yet sufficient context for downstream test synthesis.

To address these challenges, the *Intent Alignment Agent* decomposes the TDL into structured search units and performs a three-step retrieval-and-expansion process over the repository graph.

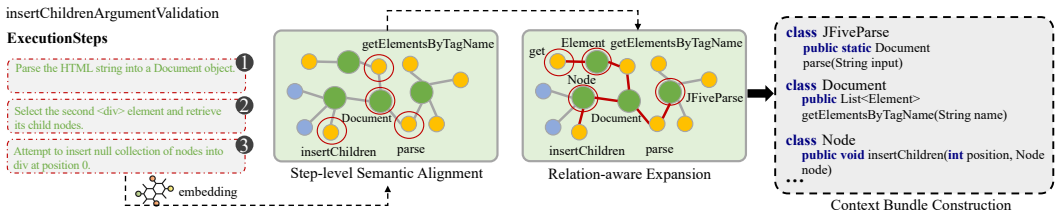


Fig. 4. Semantic alignment of TDL steps with repository graph entities, resulting in a context bundle.

(1) Step-level Semantic Querying. Each ExecutionStep in the TDL is treated as a query unit. As shown in Figure 4, in the `insertChildrenArgumentValidation` case, a step such as *Parse the HTML string into a Document* is encoded into embeddings and searched against the repository graph. We utilize *MiniLM* [9] as the embedding model because it offers a practical balance between accuracy and efficiency, enabling scalable retrieval across large repositories without sacrificing semantic precision. For each query, we perform a k-NN search over the graph index to retrieve the top-k semantically similar nodes (implemented with *FAISS* [3]). This design prioritizes recall, while subsequent agents (relation expansion and planning) filter and validate candidates to ensure executability. Retrieving multiple candidates instead of a single *best match* directly addresses the challenge of cross-language and stylistic variations, where no exact signature correspondence exists. Unlike prior approaches based on explicit API matching, this step enables broader semantic coverage while preserving retrieval efficiency. **(2) Relation-aware Expansion.** The retrieved nodes are then expanded along typed edges in the repository graph (e.g., `INVOKES`, `HAS_PARAM`, `RETURNS`). For example, retrieving `insertChildren` also pulls in its parameter class `Node` and the constructor path required to instantiate it, ensuring that the generated test satisfies all required dependencies for execution. **(3) Context Bundle Construction.** Finally, the relevant entities are assembled into a *Context Bundle*—a compact subgraph containing the minimal set of classes, methods, and dependencies required to fulfill the test intent. As shown in Figure 4, this bundle includes `JFiveParse.parse`, `Document.getElementsByTagName`, and `Node.insertChildren`, and serves as the input for subsequent test generation.

Compared with existing approaches such as *MUT* and *METALLICUS* that rely on explicit API mappings, *Intent Alignment Agent* supports multi-to-multi, cross-language semantic alignment and ensures dependency completeness via relation-aware expansion. This design allows the *INTENTTESTER* to bridge structural gaps where no API correspondence exists, enabling test migration in cases that prior approaches cannot handle.

3.5 Context-Aware Planning

The *Planning Agent* determines whether the retrieved context is sufficient for constructing an executable unit test. In typical cases, the context bundle already covers the entities required to instantiate inputs and invoke the target functionality. However, certain projects exhibit more intricate construction patterns. For example, in *NanoJSON*, initializing a JSON parser requires first constructing a `JsonTokenizer`, which itself depends on a `StringReader`; only with these chained dependencies resolved can the parser be executed correctly.

To address such cases, the agent performs a lightweight validation-and-expansion process. It first checks whether all entities required by the TDL intent (e.g., constructors, invoked methods, expected assertions) are included in the context bundle. If some dependencies are missing, the agent performs a one-hop expansion along typed edges in the repository graph (e.g., `HAS_PARAM`, `RETURNS`, `INVOKES`). If coverage remains incomplete after expansion, the intent is deemed unsupported in

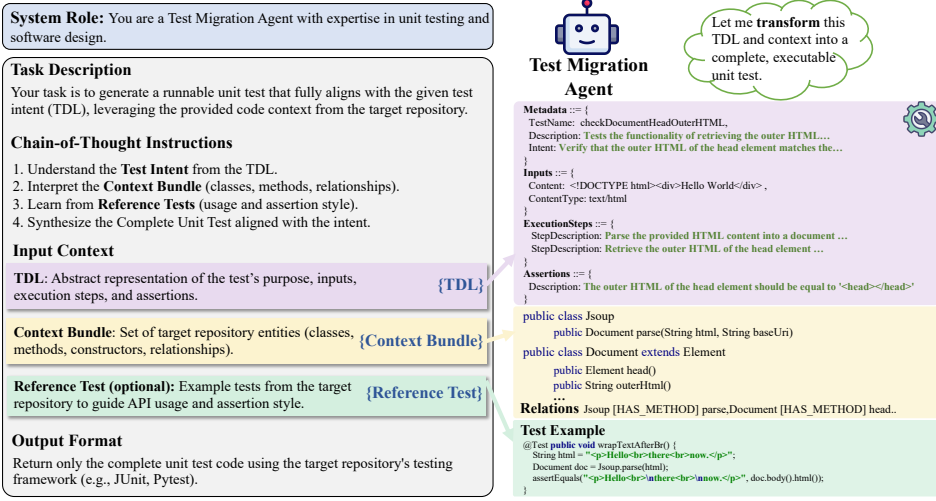


Fig. 5. Prompt Design of the Test Migration Agent, which integrates the abstracted test intent (TDL), the retrieved context bundle, and reference tests, and applies chain-of-thought prompting to synthesize complete executable unit tests in the target repository.

the target library and safely discarded. This mechanism not only avoids spurious generation when the functionality does not exist in the target repository, but also ensures that test generation proceeds only when the context is both sufficient and compact, tolerating minor retrieval gaps while preventing false migrations.

3.6 Intent-Guided Test Synthesis

The *Test Migration Agent* is responsible for constructing complete unit tests from the extracted intent and retrieved context. To enable step-wise reasoning, we design a Chain-of-Thought (CoT) prompting paradigm [23, 32], which explicitly decomposes the generation process into a sequence of reasoning steps. This CoT design mirrors the reasoning process of human testers: first clarifying the testing objective, then identifying the necessary entities and their relationships, and finally constructing executable code. As shown in Figure 5, the agent is guided through four steps: **① Understanding the test intent** as described in the TDL. **② Interpreting the Context Bundle**, including relevant classes, methods, and their relationships in the target repository. **③ Learning from reference tests** retrieved in prior steps to capture realistic API usage and assertion styles. **④ Synthesizing the complete test case**, ensuring alignment with both the functional intent and the structural constraints of the repository.

The core novelty lies in integrating functional intent (from TDL) with repository-specific usage patterns (from the Context Bundle and reference tests), enabling the *INTENTTESTER* to migrate tests that are both semantically faithful and structurally valid. Unlike prior approaches such as MUT or METALLICUS, which depend on rigid code mappings, handcrafted templates, or manual adjustments, our agent supports broader cross-library migration in a fully automated manner. This paradigm enables the reuse of tests even when explicit structural mappings are absent, thereby advancing automated test migration toward more realistic and heterogeneous software ecosystems.

Table 2. Dataset overview. For each repository, #Src = original tests in that repo; #Intent_{in} = tests *migrated into* this repo from the other two repositories in the same domain (after splitting/filtering).

Domain	Repository	Language	#Src	#Intent _{in}
JSON	Gson [4]	Java	87	160 (= NanoJSON 82 + SimpleJSON 78)
	NanoJSON [10]	Java	82	165 (= Gson 87 + SimpleJSON 78)
	SimpleJSON [12]	Python	78	169 (= Gson 87 + NanoJSON 82)
HTML	Jsoup [7]	Java	283	203 (= JFiveParse 65 + Domonic 138)
	JFiveParse [6]	Java	65	421 (= Jsoup 283 + Domonic 138)
	Domonic [2]	Python	138	348 (= Jsoup 283 + JFiveParse 65)
Time	Time4j [14]	Java	92	204 (= Threeten 71 + Maya 133)
	Threeten [13]	Java	71	225 (= Time4j 92 + Maya 133)
	Maya [8]	Python	133	163 (= Time4j 92 + Threeten 71)
Total	9 repositories		1,029	2,058

3.7 Iterative Validation and Feedback

The outputs of individual agents are not always reliable in isolation: a generated TDL may omit essential details, the retrieved context may lack key dependencies, or the synthesized test may contain incorrect assertions. To address this, INTENTTESTER integrates an iterative validation loop. Each intermediate artifact—TDL, context bundle, and migrated test—is checked for structural and semantic consistency. When inconsistencies are detected, the framework automatically feeds back error signals to the corresponding agent and triggers regeneration with a refined prompt. If errors persist after three iterations, the migration instance is discarded to prevent infinite retries, ensuring that the framework remains efficient and does not produce unreliable outputs.

This lightweight feedback loop ensures that incomplete or incorrect intermediate results do not propagate into the final output. More importantly, it reflects the practical reality of test construction: assembling an executable test often requires multiple refinements, especially under heterogeneous libraries. By embedding this iterative mechanism, INTENTTESTER improves robustness without requiring human intervention, striking a balance between automation and reliability.

4 Evaluation

Our experiments are designed to address the following research questions:

- **RQ1: What is the quality of the intent test code migrated by INTENTTESTER?**
- **RQ2: How effective are the intent tests migrated by INTENTTESTER?**
- **RQ3: How effective is TDL in generating intent tests?**
- **RQ4: How does the absence of each agent affect migration success in INTENTTESTER?**

4.1 Experimental Setup

Dataset. We collect nine widely used open-source libraries from GitHub, covering three domains: JSON processing, HTML parsing, and time manipulation, with three repositories per domain (six in Java and three in Python). This design enables evaluation in both intra-language and cross-language migration scenarios. To ensure quality and reproducibility, we require that each repository (i) compiles and runs correctly, (ii) has active commits within the last six months, and (iii) contains a sufficient number of valid test cases. We manually review and execute all tests, discarding failing

or invalid ones. In total, we obtain 1,029 source tests across the nine repositories. From these, we construct 2,058 intent tests by deriving, for each repository, intent tests from the original cases of the other two repositories in the same domain. Table 2 summarizes the domains, repositories, and the distribution of source and intent tests.

Baseline. METALLICUS [41] is a semi-automated framework for Java and Python that migrates tests by mining API-level correspondences. It extracts method signatures and documentation from the source tests, retrieves functionally similar candidates in the target repository, and adapts them using manually predefined templates. While effective in simple settings, this process depends on manual intervention when mappings are sparse or dependencies are complex. For fair comparison, we preserve its *core mapping and retrieval components*, which represents the main technical contribution of METALLICUS. To eliminate manual bias and ensure comparability with our approach, we replace its template-based adaptation with the same LLM (Llama-3.3-70B, default configuration) used in INTENTTESTER for final code generation. MUT [21] targets Java-C++ test migration through class- and API-level mappings. It constructs correspondences via signature similarity, applies hard-coded translation rules, and finally requires manual adjustments to ensure executability. To ensure comparability, we retain its mapping stage but replace the rule-based adaptation with the same LLM used in INTENTTESTER, enabling automatic completion and refinement of migrated tests.

4.2 RQ1: What is the quality of the intent test code migrated by INTENTTESTER?

To evaluate the quality of migrated tests, we examine their syntactic correctness after integrating them into the target repositories. This metric reflects whether the produced test code compiles (Java) or runs without syntax errors and unresolved references (Python), serving as the foundation for subsequent execution-based evaluation (RQ2). As shown in Table 3, INTENTTESTER produces 2,776 syntactically correct tests out of 3,257, achieving an overall correctness rate of 85%. In contrast, MUT and METALLICUS achieve only 51% and 43%, respectively. INTENTTESTER achieves superior test syntax correctness through intent migration compared to MUT and METALLICUS, demonstrating the robustness of its intent-driven design.

This advantage is attributed to three core design principles. First, the TDL abstracts test functionality from syntactic and library-specific complexities, allowing tests to be re-expressed in a portable, intent-level form. Second, the *Intent Alignment Agent* retrieves not only semantically relevant methods but also their dependent entities (e.g., parameters, constructors), ensuring dependency completeness within the Context Bundle. Third, the *Planning* and *Verification* agents filter out test intents unsupported by the target repository, preventing spurious test generation and ensuring that only valid contexts are migrated. Together, these agents jointly guarantee robustness, explaining why INTENTTESTER consistently outperforms code-mapping-based baselines.

One illustrative case comes from migrating a *Gson* test that verifies the serialization of a top-level integer into JSON. In *Gson*, this intent is expressed through the use of `JsonWriter` together with a `StringWriter`, which requires an explicit `close()` operation to finalize the output. In contrast, the equivalent behavior in Python's *SimpleJSON* relies on `json.dump` combined with `StringIO`, where the output must be rewound using `seek(0)` before reading. Although both tests validate the same intent: *serialize integer to JSON and check that the result is 123*, the APIs differ fundamentally. No direct mapping exists between `JsonWriter.close()` and `StringIO.seek(0)`, nor between `writer.value()` and `json.dump`. Consequently, code-mapping-based tools such as MUT or METALLICUS cannot establish a usable correspondence. However, INTENTTESTER abstracts the test intent into a TDL representation and semantically aligns it with the target repository. By reasoning about functional equivalence rather than API signatures, it successfully generates a valid Python test that compiles and runs correctly. This example highlights how intent-driven abstraction allows cross-language and cross-library migration even in the absence of structural mappings.

Table 3. Syntactic correctness of migrated intent tests (filter. = tests after filtering, cor. = syntactically correct tests, rate = correctness rate).

Domain	Repository	#Tests	INTENTTESTER			MUT		METALLICUS	
			filter.	cor.	rate	cor.	rate	cor.	rate
JSON	Gson	534	416	367	88%	295	55%	225	42%
	NanoJSON	624	340	252	74%	190	50%	89	26%
	SimpleJSON	685	650	634	97%	342	50%	311	48%
HTML	Jsoup	436	242	217	90%	160	54%	154	63%
	JFiveParse	991	398	288	72%	230	49%	167	42%
	Domonic	959	311	247	79%	165	53%	106	34%
Time	Time4j	412	222	168	76%	120	52%	98	44%
	Threeten	411	338	326	96%	200	49%	124	37%
	Maya	484	340	277	82%	178	52%	135	40%
Total		5,536	3,257	2,776	85%	1,880	51%	1,409	43%

Although correctness is substantially improved, INTENTTESTER still encounters errors in a subset of tests. The dominant failure mode arises from complex parameter construction, where the target API requires unusually deep initialization chains—often violating best-practice design principles. For example, in *Time4j*, the method `MultiFormatParser.parse()` expects a `ChronoFormatter` object that depends on seven layers of nested reference types. Similarly, in *JFiveParse*, constructing a `NodeMatchers` instance requires traversing a deeply coupled dependency graph. In such cases, when the retrieved Context Bundle does not capture all transitive dependencies, the migrated test fails compilation due to unresolved constructors. These failures are rare and concentrated in repositories with atypically deep constructor graphs. This RQ shows that intent-driven test reuse achieves much higher syntactic correctness than code-mapping baselines. By leveraging TDL abstraction and Context Bundle reasoning, INTENTTESTER consistently generates compilable tests across heterogeneous repositories.

4.3 RQ2: How effective are the intent tests migrated by INTENTTESTER?

4.3.1 Overall Effectiveness. We evaluate the effectiveness of intent-based test migration by executing all syntactically correct tests migrated in RQ1 across nine repositories. As shown in Table 4, INTENTTESTER achieves an overall 72% pass rate, with consistent performance across domains: JSON (73%), HTML (71%), and Time (71%). To further measure how effectively the migrated tests preserve intent rather than just compiling, we define Effective Accuracy as: $Effective\ Accuracy = \frac{Pass\ Tests}{Filtered\ Tests}$, where Filtered Tests are those that passed the syntactic check in RQ1. This metric reflects the fraction of intent tests that both compile and run correctly, thereby preserving their intended functionality. By this measure, INTENTTESTER attains 82% in JSON and 68% in both HTML and Time, confirming its ability to generate runnable and meaningful cross-repository tests.

Unlike baselines such as MUT and METALLICUS, which report only 51% and 43% syntactic correctness in RQ1, INTENTTESTER advances further to provide end-to-end executable tests. Baselines often require additional manual adaptation before execution (e.g., re-specifying parameters, re-writing missing oracles, or handling untranslated original APIs in tests due to missing mappings), which limits their automation potential.

Table 4. Execution results of intent tests in target repositories. Pass and Fail counts are reported with percentages; failures are categorized into F1–F4.

Domain	Pass	Fail	Failure Categories				Accuracy
			F1	F2	F3	F4	
JSON	916 (73%)	337 (27%)	105	198	15	19	82%
HTML	531 (71%)	221 (29%)	106	97	8	10	68%
Time	549 (71%)	222 (29%)	155	52	2	13	68%
Total	1,996 (72%)	780 (28%)	366	347	25	42	74%

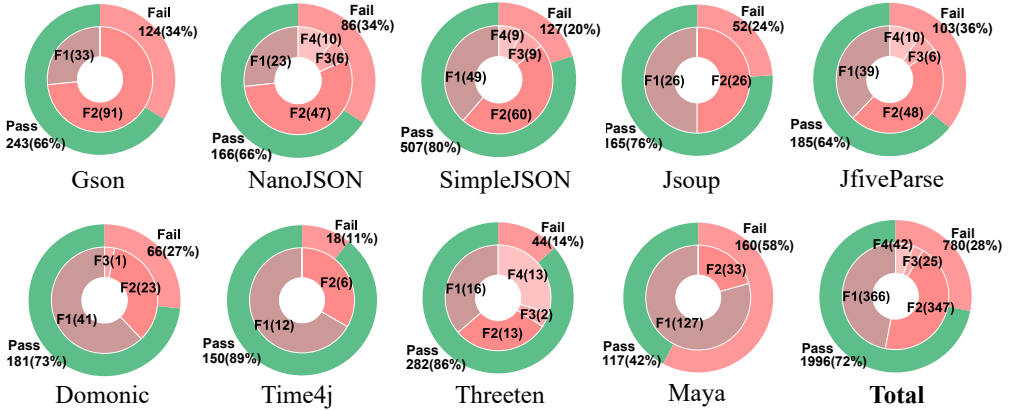


Fig. 6. Distribution of execution outcomes for migrated intent tests across different repositories, showing the proportion of successful and failed runs.

4.3.2 Failure Categorization. We next analyze the failing cases to understand their root causes. Execution outcomes are divided into **Pass** (successful execution) and **Fail** (execution failure). For the latter, we conduct a fine-grained classification into four categories: ② **F1 Invalid Adaptation** – failures caused by misuse of APIs or parameter mismatches despite syntactic correctness. ③ **F2 Design Differences** – cross-library divergences in semantics or API behavior, e.g., normalization policies or indexing conventions. ④ **F3 Defect Discovery** – actual defects uncovered in the target repository, such as null dereference or infinite recursion. ⑤ **F4 Feature Gaps** – functionality supported in the source repository but absent in the target, though potentially implementable.

As summarized in Table 4, F1 and F2 account for most failures (over 90%), reflecting expected challenges of adaptation and heterogeneous library design rather than flaws in INTENTTESTER. Importantly, 25 failures correspond to real defects (F3), several of which have been acknowledged and fixed by maintainers, while 42 cases fall into F4, highlighting opportunities for feature enhancement.

Figure 6 presents the distribution of execution outcomes across all nine repositories. The JSON domain achieves the highest effective test rate at 82%, while the HTML and Time domains both reach 68%. This indicates that INTENTTESTER reliably reuses tests across domains with distinct design styles. Most failures fall into F1 Invalid Adaptation (47%) and F2 Design Differences (45%). Importantly, these are expected outcomes in cross-library migration rather than intrinsic limitations of INTENTTESTER. By contrast, F3 Defect Discovery and F4 Feature Gaps collectively account for 9% of failures but contribute the most value, exposing actual bugs and surfacing missing functionality.

Table 5. Classification of real defects uncovered by INTENTTESTER. Each category includes representative error types.

Category	Representative Error Types	Count
HTML/XML Parsing	Missing structure validation; incorrect DOM node handling	4
JSON Parsing	Unicode misparsing; failure on deeply nested JSON; precision loss	7
Date/Time Handling	Incorrect format conversion; date calculation mismatch	2
Data Validation	Unhandled null; generation of invalid HTML tags	3
Exception Handling	Null pointer dereference; infinite recursion → stack overflow	6
Format Conversion	BigInt mis-conversion; wrong type casting	3
Total		25

4.3.3 Case Study and Defect Categorization. F1 Invalid Adaptation. In *JFiveParse*, an intent test fails when attempting to retrieve the root DOM element. The migrated test uses `getFirstChild`, which syntactically compiles but semantically retrieves the wrong entity. This results in an execution failure even though the test is valid at the code level. Such cases highlight that semantic API discrepancies remain a key challenge in cross-library test migration, but they also show where INTENTTESTER could improve by enriching its context model with usage constraints.

F2 Design Differences. Intent tests also expose subtle divergences in functionality. For example, *Jsoup* automatically normalizes HTML tags (e.g., `<DIV>` → `<div>`), while *JFiveParse* preserves casing. Similarly, in *Domonic*, calling `get(-1)` returns the last element, whereas *Jsoup* enforces strict index validation. These are not bugs but provide actionable insights for developers migrating or co-maintaining code across libraries.

Beyond adaptation challenges, a subset of execution failures revealed critical findings. As summarized in Table 5, INTENTTESTER uncovered 25 genuine defects across JSON, HTML, and Time repositories, including incorrect handling of deeply nested JSON, missing validation of HTML nodes, unhandled null values, and stack overflows in recursive serialization. We reported these issues to maintainers, of which six have been acknowledged and three already patched. In contrast, baselines such as MUT and METALLICUS could not uncover such defects, since their migrated tests failed to execute reliably. In addition, 42 cases fall under feature gaps, where tests exercised functionality absent from the target library but potentially useful (e.g., automatic HTML normalization in *JFiveParse*). Unlike baselines, which fail to reach execution due to missing mappings or incomplete contexts, INTENTTESTER produces runnable tests that expose semantic mismatches across libraries. Overall, F1–F2 represent adaptation and heterogeneity challenges, while F3–F4 highlight the added value of INTENTTESTER: identifying real defects and surfacing enhancement opportunities. These findings demonstrate that the value of INTENTTESTER extends beyond raw pass rates: intent-driven tests enable cross-library semantic validation, uncover real defects, and surface meaningful opportunities for enhancement.

4.4 RQ3: How effective is TDL in generating intent tests?

Since TDL is the core abstraction in our framework, it is essential to evaluate whether it faithfully captures test intent. Note that the impact of TDL on migration success is further examined in RQ4 through ablation studies. Here, we focus on two complementary perspectives: automated reconstruction fidelity and human validation.

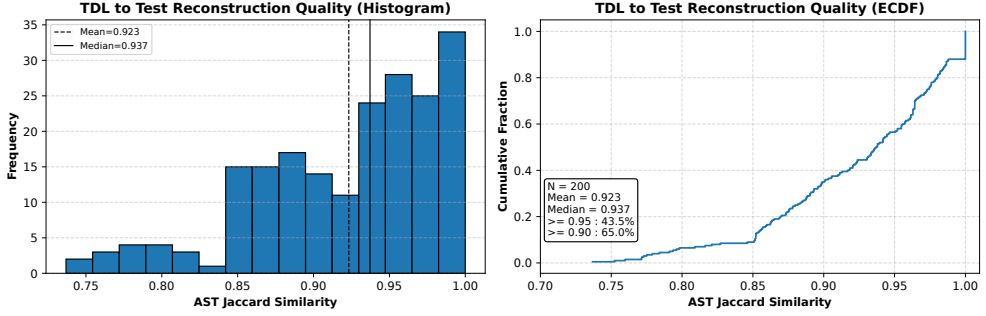


Fig. 7. Reconstruction quality from TDL: histogram (left) and ECDF (right) of AST Jaccard similarity, evidencing high fidelity; lower scores mainly reflect LLM refinements rather than semantic loss.

4.4.1 Reconstruction study. To evaluate whether TDL serves as a faithful abstraction of test intent, we conduct a reconstruction study. We randomly sample 200 tests from nine repositories and generate their corresponding TDL representations. Given the TDL and source repository context, an LLM is asked to reconstruct the original test. We evaluate reconstruction fidelity by comparing the original test with the TDL-based reconstructed version using AST-based Jaccard Similarity. This metric reflects whether the TDL contains sufficient information to faithfully rebuild the original test, beyond surface-level variations such as identifier renaming.

As shown in Figure 7, the reconstructed tests achieve consistently high similarity, with a mean of 0.92 and a median of 0.94. Nearly two-thirds of the cases exceed 0.90, and more than 40% reach above 0.95, demonstrating that TDL preserves sufficient structural information to regenerate the original tests. Further analysis reveals that cases with lower similarity do not indicate semantic loss, but rather LLM-driven refinements. For example, some reconstructed tests add descriptive messages to assertion statements, or replace `assertEqual` with semantically equivalent `assertTrue`. In exception handling, the model sometimes rewrites `assertThrows` as `@Test(expected=...)`, changing syntax but not behavior. These adjustments reduce structural similarity while maintaining the test’s functional intent. TDL provides a compact and language-agnostic abstraction that enables faithful reconstruction of tests. Minor structural divergences reflect alternative, but valid, realizations of the same intent, underscoring TDL’s suitability as the foundation for cross-library test migration.

4.4.2 Human Validation. To assess whether TDL faithfully captures test intent, we conduct a user study with three senior developers who each have over five years of experience in both Java and Python. From nine projects, we randomly select fifty TDL–test pairs and ask participants to rate TDL quality on four dimensions (Readability, Completeness, Intent, Reusability) on a 5-point Likert scale. Figure 8 shows the results: 80% of readability scores are 4 or above, 80% for completeness, 78% for intent fidelity, and 84% for reusability (with nearly half of the reusability scores rated 5). These outcomes suggest that TDL is generally clear, preserves the intent of the original test, and supports reuse across

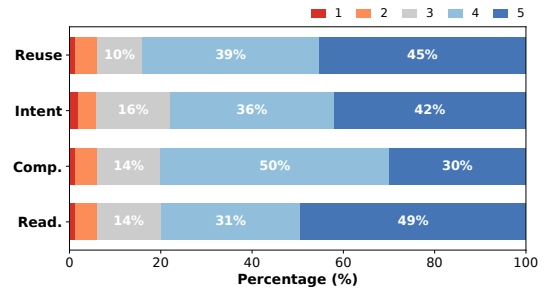


Fig. 8. Results of the user study on TDL.

repositories and languages. Overall, TDL provides a reliable abstraction of test intent, capturing both the verification goal and sufficient structural information for cross-library reuse.

Error Analysis. We manually inspect the low-scoring cases (ratings of 3 or below) and identify two recurring issues. First, some descriptions are over-generalized, omitting key details such as exception types—for example, describing a step as *verify error handling* without specifying the expected `IllegalArgumentException`, which reduces completeness. Second, some TDLs are overly verbose, using long phrases for simple operations (e.g., *initialize an empty collection object and then proceed to append*) instead of concise ones (e.g., *create an empty list*), which reduces readability. Both issues are minor, do not affect the correctness of subsequent test migration, and can be mitigated by refining the prompting strategy. Importantly, none of the developers report systematic misrepresentation of test intent, indicating that the errors concern expression quality rather than semantic accuracy.

4.5 RQ4: Ablation Study.

To assess the contribution of each agent, we conduct an ablation study on a random sample of 200 tests. We measure two metrics: (1) Compilation Success Rate, i.e., whether the migrated tests are syntactically valid and compilable, and (2) Execution Pass Rate, i.e., whether the tests execute successfully on the target repositories. We compare `INTENTTESTER` against four ablated variants: *w/o Intent Abtractor Agent (TDL)* – directly using raw source tests as queries, without intent-level abstraction. *w/o Relation-aware Expansion* – performing only step-level semantic search, without dependency expansion. *w/o Planning Agent* – skipping sufficiency validation and directly generating tests. *w/o Verification Agent* – omitting post-generation checks, accepting tests without filtering.

As shown in Table 6, all components are critical. Removing TDL abstraction or Relation-aware Expansion leads to the largest drops (CSR ↓28% and 12%, EPR ↓18% and 15%), underscoring that intent-level representation and dependency modeling are key for retrieving executable contexts. Omitting Planning or Verification mainly affects robustness, with CSR falling to 62–68% (vs. 88%), since many generated tests contain unresolved symbols or inconsistent assertions that would otherwise be filtered. Overall, the agents complement each other: Intent Abstraction and Relation Expansion secure semantic and structural fidelity, while Planning and Verification ensure syntactic and execution-level reliability. Together, they enable `INTENTTESTER` to reach the highest CSR (88%) and EPR (72%), confirming the necessity of the multi-agent design for robust, executable migration.

Table 6. Ablation study on 200 randomly sampled tests, CSR: Compilation Success Rate, EPR: Execution Pass Rate.

Variant	CSR	EPR
w/o Intent Abtractor Agent	60%	54%
w/o Relation-aware Expansion	76%	57%
w/o Planning Agent	62%	49%
w/o Verification Agent	68%	51%
INTENTTESTER (Full)	88%	72%

5 Discussion

5.1 Coverage Implications of Intent-Based Test Migration

A natural question is whether intent-driven test migration with `INTENTTESTER` improves code coverage. We measured line and branch coverage before and after migration across nine repositories in three domains (Figure 9). The results show modest increases—for instance, *JFiveParse* improves by +17% in line coverage, and *Demonic* by +8%—while mature repositories such as *Jsoup* and *Gson* show only marginal gains (1–3%).

This trend is expected given the motivation of INTENTTESTER. Importantly, the design of INTENTTESTER is not to maximize structural coverage, but rather to **reuse existing, human-written tests that encode domain knowledge across repositories**. Many source tests exercise core behaviors such as parsing, serialization, or argument validation—scenarios already well-covered in mature target repositories. Consequently, migrating such tests rarely introduces additional coverage, but it still strengthens the test suite by (1) verifying semantic consistency across implementations, (2) uncovering previously untested failure modes (e.g., `NullPointerException` in *JFiveParse*, `StackOverflowError` in *NanoJSON*), and (3) enriching the diversity of assertion oracles.

Moreover, coverage gaps often arise from repository-specific features with no functional counterpart in the source repository. For example, *JFiveParse* contains `TokenizerTagStates`, a specialized state-machine component for HTML tokenization not found in *Jsoup* or *Domonic*. Since no equivalent test intent exists, migrating tests cannot improve coverage in such regions.

Overall, the primary contribution of INTENTTESTER lies in enabling cross-library test reuse and exposing hidden robustness issues, rather than simply boosting raw coverage metrics. Future work could integrate intent reuse with coverage-guided generation to balance semantic alignment with structural exploration.

5.2 Threats to Validity

Internal Validity. A possible threat lies in the experimental setup, such as the choice of benchmarks and sampling strategy. We mitigate this by using nine diverse open-source libraries across three domains and two programming languages, and by reporting aggregated results over thousands of generated tests. Another internal threat is in the human evaluation of TDL fidelity, which could introduce subjectivity. To reduce bias, we employed multiple senior developers, used a structured Likert-scale rubric, and measured agreement across raters. Finally, our implementation relies on a specific *Llama-3.3-70B*. While the framework itself is model-agnostic, different LLMs may vary in their ability to capture domain semantics, which could affect absolute performance numbers. We mitigate this by reporting results across diverse repositories and ensuring that all baselines use the same model configuration for fairness.

External Validity. Our current evaluation is limited to Java and Python projects in three domains (JSON, HTML, and Time). While these domains are representative and cover both intra-language and cross-language migration, future work should explore additional ecosystems (e.g., C++, JavaScript) and industrial settings. Finally, while we observed real defect discovery and feature gaps, the scale of issue reporting remains limited; broader collaboration with maintainers would further validate practical utility.

6 Related Work

API Mapping and Test Migration. Early work explored API mapping through call graph and structural analysis [20, 26, 27, 40, 43, 53, 55]. MUT [21] migrated unit tests by structural code

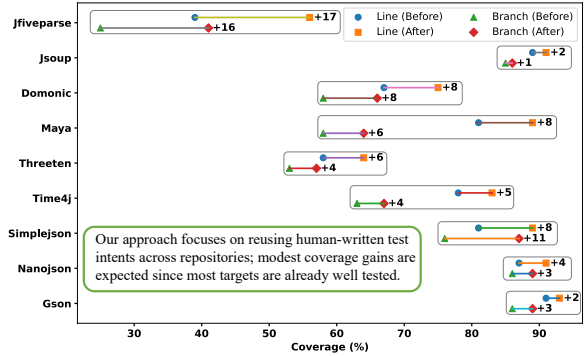


Fig. 9. Line and branch coverage achieved by INTENTTESTER across target repositories, showing the additional coverage contributed by migrated intent tests.

mapping, while Nguyen et al. [37] applied statistical learning for cross-language method mapping. METALLICUS [41] mined similar methods for automated test adaptation, and JTestMigrator [29] leveraged semantic similarity, achieving a 73% success rate in Java ecosystems. However, these approaches primarily rely on syntactic translation, often failing to preserve original test intents when semantics diverge.

Beyond GUI-Centric Migration. Much prior work targeted GUI applications [17–19, 24, 30, 33, 36, 38, 52, 54]. TestMig [38] translated cross-platform UI events, while TEMdroid [51] achieved 76% widget-matching accuracy using BERT embeddings. Recent advances in GUI semantics [34, 46, 49, 51] highlight progress in intent preservation at the UI level. In contrast, repository functionality tests across programming languages remain underexplored. Although MUT [21] systematized unit test migration, it inherited the limitations of syntactic mapping. LLM-augmented techniques, such as AutoCodeRover [50], demonstrate promise in program repair through AST analysis, but naively applied LLMs often generate semantically inconsistent tests when bridging library gaps.

7 Conclusion and Future Work

This paper presents INTENTTESTER, a multi-agent framework for intent-driven test migration that combines TDL abstraction, repository graphs, and LLM-guided synthesis. By shifting from brittle structural mappings to semantic alignment of test intent, INTENTTESTER enables automated cross-library and cross-language test migration without manual intervention. Our evaluation on nine real-world projects shows that INTENTTESTER achieves 85% syntactic correctness and 74% execution success, while also uncovering 25 real defects across widely used libraries. These results demonstrate both its practical effectiveness and its ability to strengthen test suites with minimal developer effort. For future work, we plan to extend INTENTTESTER to support additional programming languages and library ecosystems, and to investigate how different LLM sizes and configurations influence migration quality. This will further broaden its applicability and robustness in diverse software environments.

8 Data Availability

Our tool is available at [5].

Acknowledgments

This work was supported by the National Key R&D Program of China (No. 2024YFB4506400).

References

- [1] 2026. Antlr. <https://www.antlr.org/>.
- [2] 2026. Domonic. <https://github.com/byteface/domonic>.
- [3] 2026. FAISS. <https://github.com/facebookresearch/faiss>.
- [4] 2026. Gson. <https://github.com/google/gson>.
- [5] 2026. IntentTester. <https://github.com/testmigrator/intenttest>.
- [6] 2026. Jfiveparse. <https://github.com/digitalfondue/jfiveparse>.
- [7] 2026. jsoup. <https://github.com/jhy/jsoup>.
- [8] 2026. Maya. <https://github.com/kennethreitz/maya>.
- [9] 2026. MiniLM. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>.
- [10] 2026. Nanojson. <https://github.com/mmastrac/nanojson>.
- [11] 2026. Neo4j. <https://neo4j.com/>.
- [12] 2026. Simplejson. <https://github.com/simplejson/simplejson>.
- [13] 2026. Threeten. <https://github.com/ThreeTen/threetenbp>.
- [14] 2026. Time4j. <https://github.com/MenoData/Time4j>.
- [15] Maurício Aniche, Christoph Treude, and Andy Zaidman. 2021. How developers engineer test cases: An observational study. *IEEE Transactions on Software Engineering* 48, 12 (2021), 4925–4946. doi:10.1109/TSE.2021.3129889

- [16] Baris Ardic, Carolin Brandt, Ali Khatami, Mark Swillus, and Andy Zaidman. 2025. The qualitative factor in software testing: A systematic mapping study of qualitative methods. *Journal of Systems and Software* (2025), 112447. doi:10.1016/J.JSS.2025.112447
- [17] Farnaz Behrang and Alessandro Orso. 2018. Test migration for efficient large-scale assessment of mobile app coding assignments. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 164–175. doi:10.1145/3213846.3213854
- [18] Farnaz Behrang and Alessandro Orso. 2019. Test migration between mobile apps with similar functionality. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 54–65. doi:10.1109/ASE.2019.00016
- [19] Benyamin Beyzaei, Saghar Talebipour, Ghazal Rafiei, Nenad Medvidović, and Sam Malek. 2025. Automated Test Transfer across Android Apps using Large Language Models. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 2227–2250. doi:10.1145/3728975
- [20] Zirui Chen, Xing Hu, Xin Xia, and Xiaohu Yang. 2026. Every Maintenance Has Its Exemplar: The Future of Software Maintenance through Migration. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.48550/ARXIV.2602.14046
- [21] Yi Gao, Xing Hu, Tongtong Xu, Xin Xia, David Lo, and Xiaohu Yang. 2024. MUT: Human-in-the-Loop Unit Test Migration. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12. doi:10.1145/3597503.3639124
- [22] Yi Gao, Xing Hu, Xiaohu Yang, and Xin Xia. 2025. Automated unit test refactoring. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 713–733. doi:10.1145/3715750
- [23] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79. doi:10.1145/3695988
- [24] Gang Hu, Linjie Zhu, and Junfeng Yang. 2018. AppFlow: using machine learning to synthesize robust, reusable UI tests. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 269–282. doi:10.1145/3236024.3236055
- [25] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90. doi:10.1007/S10664-022-10131-8
- [26] Zhenfei Huang, Junjie Chen, Jiajun Jiang, Yihua Liang, Hanmo You, and Fengjie Li. 2024. Mapping APIs in Dynamic-typed Programs by Leveraging Transfer Learning. *ACM Transactions on Software Engineering and Methodology* 33, 4 (2024), 1–29. doi:10.1145/3641848
- [27] Mohayeminul Islam, Ajay Kumar Jha, Ildar Akhmetov, and Sarah Nadi. 2024. Characterizing Python Library Migrations. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 92–114. doi:10.1145/3643731
- [28] Ajay Kumar Jha, Mohayeminul Islam, and Sarah Nadi. 2023. Jtestmigbench and jtestmigtax: A benchmark and taxonomy for unit test migration. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 713–717. doi:10.1109/SANER56733.2023.00077
- [29] Ajay Kumar Jha and Sarah Nadi. 2024. Migrating Unit Tests Across Java Applications. In *2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 131–142. doi:10.1109/SCAM63643.2024.00022
- [30] Farideh Khalili, Leonardo Mariani, Ali Mohebbi, Mauro Pezzè, and Valerio Terragni. 2024. Semantic matching in GUI test reuse. *Empirical Software Engineering* 29, 3 (2024), 70. doi:10.1007/S10664-023-10406-8
- [31] Ali Khatami and Andy Zaidman. 2023. Quality assurance awareness in open source software projects on GitHub. In *2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 174–185. doi:10.1109/SCAM59687.2023.00027
- [32] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–23. doi:10.1145/3690635
- [33] Jun-Wei Lin, Reyhaneh Jabbarvand, and Sam Malek. 2019. Test transfer across mobile apps through semantic mapping. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 42–53. doi:10.1109/ASE.2019.00015
- [34] Shuqi Liu, Yu Zhou, Tingting Han, and Taolue Chen. 2022. Test reuse based on adaptive semantic matching across android mobile applications. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 703–709. doi:10.1109/QRS57517.2022.00076
- [35] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2024. How to understand whole software repository? *arXiv preprint arXiv:2406.01422* (2024). doi:10.48550/ARXIV.2406.01422
- [36] Leonardo Mariani, Ali Mohebbi, Mauro Pezzè, and Valerio Terragni. 2021. Semantic matching of gui events for test reuse: are we there yet?. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 177–190. doi:10.1145/3460319.3464827

- [37] Anh Tuan Nguyen, Hoan Anh Nguyen, Tung Thanh Nguyen, and Tien N Nguyen. 2014. Statistical learning approach for mining API usage mappings for code migration. In *Proceedings of the 29th ACM/IEEE international conference on Automated software engineering*. 457–468. doi:10.1145/2642937.2643010
- [38] Xue Qin, Hao Zhong, and Xiaoyin Wang. 2019. Testmig: Migrating gui test cases from ios to android. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 284–295. doi:10.1145/3293882.3330575
- [39] Max Schafer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. doi:10.1109/TSE.2023.3334955
- [40] Yanjie Shao, Tianyue Luo, Xiang Ling, Limin Wang, and Senwen Zheng. 2022. Cross Platform API Mappings based on API Documentation Graphs. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 926–935. doi:10.1109/QRS57517.2022.00097
- [41] Devika Sondhi, Mayank Jobanputra, Divya Rani, Salil Purandare, Sakshi Sharma, and Rahul Purandare. 2021. Mining similar methods for test adaptation. *IEEE Transactions on Software Engineering* 48, 7 (2021), 2262–2276. doi:10.1109/TSE.2021.3057163
- [42] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2024. Chatgpt vs sbst: A comparative assessment of unit test suite generation. *IEEE Transactions on Software Engineering* (2024). doi:10.1109/TSE.2024.3382365
- [43] Cédric Teyton, Jean-Rémy Falleri, and Xavier Blanc. 2013. Automatic discovery of function mappings between similar libraries. In *2013 20th Working Conference on Reverse Engineering (WCRE)*. IEEE, 192–201. doi:10.1109/WCRE.2013.6671294
- [44] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* (2024). doi:10.1109/TSE.2024.3368208
- [45] Ying Wang, Bihuan Chen, Kaifeng Huang, Bowen Shi, Congying Xu, Xin Peng, Yijian Wu, and Yang Liu. 2020. An empirical study of usages, updates and risks of third-party libraries in java projects. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 35–45. doi:10.1109/ICSME46990.2020.00014
- [46] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2024. Intent-driven mobile gui testing with autonomous large language model agents. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 129–139. doi:10.1109/ICST60714.2024.00020
- [47] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726. doi:10.1145/3660783
- [48] Junwei Zhang, Xing Hu, Xin Xia, Shing-Chi Cheung, and Shanping Li. 2026. Automated Unit Test Generation via Chain-of-Thought Prompt and Reinforcement Learning from Coverage Feedback. *ACM Transactions on Software Engineering and Methodology* 35, 4 (2026), 1–30.
- [49] Yakun Zhang, Chen Liu, Xiaofei Xie, Yun Lin, Jin Song Dong, Dan Hao, and Lu Zhang. 2024. LLM-based Abstraction and Concretization for GUI Test Migration. *arXiv preprint arXiv:2409.05028* (2024). doi:10.48550/ARXIV.2409.05028
- [50] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604. doi:10.1145/3650212.3680384
- [51] Yakun Zhang, Wenjie Zhang, Dezhi Ran, Qihao Zhu, Chengfeng Dou, Dan Hao, Tao Xie, and Lu Zhang. 2024. Learning-based widget matching for migrating gui test cases. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3623322
- [52] Yakun Zhang, Qihao Zhu, Jiwei Yan, Chen Liu, Wenjie Zhang, Yifan Zhao, Dan Hao, and Lu Zhang. 2024. Synthesis-Based Enhancement for GUI Test Case Migration. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 869–881. doi:10.1145/3650212.3680327
- [53] Zejun Zhang, Minxue Pan, Tian Zhang, Xinyu Zhou, and Xuandong Li. 2020. Deep-diving into documentation to develop improved java-to-swift api mapping. In *Proceedings of the 28th International Conference on Program Comprehension*. 106–116. doi:10.1145/3387904.3389282
- [54] Yixue Zhao, Justin Chen, Adriana Sejfia, Marcelo Schmitt Laser, Jie Zhang, Federica Sarro, Mark Harman, and Nenad Medvidovic. 2020. Fruiter: a framework for evaluating ui test reuse. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1190–1201. doi:10.1145/3368089.3409708
- [55] Bingzhe Zhou, Xinying Wang, Shengbin Xu, Yuan Yao, Minxue Pan, Feng Xu, and Xiaoxing Ma. 2023. Hybrid API migration: A marriage of small API mapping models and large language models. In *Proceedings of the 14th Asia-Pacific Symposium on Internetware*. 12–21. doi:10.1145/3609437.3609466

Received 2025-09-03; accepted 2025-12-22